

The Cost of Centralised Leadership in Distributed Consensus

An Implementation-Level Measurement Study of Leader-Based and Leaderless Consensus

Sephy Ryu
Independent Researcher
South Korea

Abstract

Leader-based consensus protocols such as Raft route write requests through an elected leader, whereas leaderless protocols such as EPaxos permit proposal initiation at multiple replicas. This paper presents an implementation-level comparison of these approaches under a common benchmark methodology: the same workload generator, allocations, failure injection, and metrics, with ten repetitions across seven experiment families and replica counts from three to nine on a single containerised host.

In the primary implementations, EPaxos achieved higher median throughput than Raft at three and five replicas (ratios 1.22 and 1.73, exact Mann-Whitney $p < 0.001$), but no significant difference was detected at seven or nine replicas. The EPaxos decline at those sizes is associated with request timeouts at the 2,000 ms client timeout, while successful requests completed in 5.4–5.6 ms, the same as at three replicas. Median latency was flat for EPaxos across replica counts while Raft grew from 6.9 to 14.6 ms, and at concurrency 256 EPaxos measured 41,077 req/s against 19,333 req/s for Raft. Losing the Raft leader produced availability gaps of 1.81–4.74 s, whereas losing a Raft follower or an EPaxos replica produced gaps below 0.1 s.

Evaluating two Raft and two EPaxos implementations shows that implementation choice materially affects performance: under 10 ms of injected one-way delay, throughput fell by 72–96 % across implementations, and the etcd Raft implementation collapsed by 94 % at 1 ms, reversing the baseline ordering. A correctness harness passed 18 deterministic state-convergence tests across all implementations. The results are controlled measurements of the evaluated implementations, not universal properties of Raft or EPaxos.

Keywords

consensus, Raft, EPaxos, leader-based ordering, leaderless ordering, benchmarking

1 Introduction

Distributed systems replicate state across machines to provide fault tolerance, and consensus protocols coordinate the replicas so that their states remain consistent. Two design families dominate practice. Leader-based protocols such as Raft [6] direct client operations through an elected leader, which orders log entries and coordinates replication; leader failure requires an election before processing resumes. Leaderless protocols such as EPaxos [5] allow any replica to propose operations and determine dependencies among them, removing

the single ordering point at the cost of dependency tracking and conflict detection.

These architectural differences motivate an empirical comparison, but a simple protocol-level ranking is hard to interpret. Measured performance depends on replica count, concurrency, conflict rate, and communication delay, and it is affected by implementation mechanisms such as persistence, scheduling, and message handling. A difference between two implementations therefore cannot automatically be attributed to the consensus algorithm alone. This is particularly important for leader-based versus leaderless comparisons: the presence of a leader is a structural difference, but it does not by itself determine the throughput or latency of a concrete implementation.

In this paper we experimentally compare Raft and EPaxos as representative leader-based and leaderless protocols, evaluating throughput, latency, resource utilisation, and failure behaviour under controlled variations in replica count, concurrency, conflict rate, read/write composition, and communication conditions. We additionally evaluate two independent implementations of each protocol family to quantify how much of the observed difference is implementation-level rather than protocol-level.

We focus on the following research questions:

- **RQ1:** How do the measured throughput and latency of the evaluated leader-based and leaderless implementations change with replica count, client concurrency, conflict rate, and read/write composition?
- **RQ2:** How do the evaluated implementations behave under replica and leader failures, and under injected communication delay?
- **RQ3:** How much of the measured performance difference between Raft and EPaxos is attributable to the concrete implementation rather than to the consensus algorithm family?

We make the following contributions:

- **Reproducible benchmark infrastructure:** the Consensus Lab executes Raft and EPaxos in a common containerised environment through a common client harness and benchmark-facing interface.
- **Sensitivity-oriented evaluation:** throughput and latency are measured across controlled variations in replica count, concurrency, conflict rate, read/write composition, and communication conditions rather than at a single configuration.
- **Failure behaviour analysis:** we characterise availability under Raft leader failure, Raft follower failure, and EPaxos replica failure.

- **Implementation-level analysis:** two implementations of each protocol family are evaluated to quantify the effect of implementation choices on the measured performance relationship.
- **Open infrastructure:** the lab, configurations, and analysis pipeline are released at <https://github.com/Sephya314/Leader-consensus-comparison>.

The experiments run in a local containerised environment, so the results are measurements of the evaluated implementations under the specified conditions and should not be read as universal production-performance estimates. The remainder of this paper is organised as follows. Section 2 reviews leader-based and leaderless consensus and related comparative studies. Section 3 describes the Consensus Lab, the implementations, and the experiment suite. Section 4 presents the results. Section 5 interprets them with respect to the research questions. Section 6 discusses threats to validity, and Section 7 summarises the findings.

2 Background and Related Work

2.1 Leader-Based and Leaderless Consensus

Consensus protocols coordinate replicas so that a replicated state machine applies operations in a consistent order despite failures. Protocols differ in how they establish this order. Leader-based protocols such as Raft [6] designate one replica as the leader: client requests are directed to it, it appends operations to its log and replicates them to followers, and an entry is committed once a quorum has acknowledged it. This centralises the write path around a single replica and makes leader failure require an election before processing resumes. Paxos [4] established the foundational agreement framework, but its abstract presentation leaves substantial implementation decisions to system designers, so systems implementing the same protocol family can differ considerably in execution path, storage, and communication behaviour.

Leaderless protocols such as EPaxos [5] allow any replica to propose operations and determine dependencies among them. Commands that do not conflict can commit without a single global ordering point, at the cost of dependency tracking and conflict detection. Removing the single ordering authority does not eliminate coordination: replicas must agree on dependencies and execute conflicting operations in a compatible order. Leaderless operation also changes failure behaviour: the loss of one replica does not require electing a replacement leader before other replicas continue proposing.

2.2 Related Work

Comparative studies of consensus designs have reached different conclusions depending on the systems and workloads examined. Howard and Mortier found that the choice of implementation and configuration often matters more than the protocol family itself when comparing Paxos and Raft [3]. van Renesse et al. showed that the relative performance of Paxos

and leaderless replication depends on the workload, particularly on command conflict [9]. Tollman et al. re-evaluated EPaxos under geo-replicated workloads and found that the original benchmarking approach did not expose the performance impact of conflicting commands, with substantially worse tail latency than the reported median [7]. These results motivate evaluating multiple independent implementations of each protocol family under controlled conditions, as we do here.

Our study compares two Raft implementations (HashiCorp Raft and etcd Raft) with two EPaxos implementations (the original `efficient/epaxos` and `nvanbenschoten/epaxos`) while varying replica count, concurrency, communication delay, and failure conditions. The comparison therefore evaluates concrete implementations under a controlled workload rather than treating the results as protocol-independent properties of Raft or EPaxos. This distinction is reflected throughout the paper and in the threats to validity discussed in Section 6.

3 Experimental Setup

3.1 Consensus Lab Architecture

The experiments are conducted with the Consensus Lab, a reproducible benchmark infrastructure that runs the evaluated implementations in a common containerised environment. Each run is an independent Docker Compose project containing one coordination service (the upstream EPaxos master [5] or a lab-provided Raft master), one container per replica, and one benchmark client. All containers share the same runtime image and the configured CPU and memory allocations.

Both protocols expose the same benchmark-facing request interface based on the upstream EPaxos `genericmr` wire protocol (`PROPOSE` $\rightarrow$ `ProposeReplyTS`). A single client binary and workload generator are used for both protocols, so the operation mix, key-selection process, concurrency control, and request-generation logic are shared. Request routing differs by protocol architecture: Raft requests are directed to the current leader reported by the raft master, whereas EPaxos requests are distributed round-robin across replicas. Reads are submitted through the consensus path in both protocols; no Raft-specific local-read optimisation is enabled.

3.2 Implementation

Each protocol is represented by two independent implementations. The lab adapters add only orchestration, monitoring, and the communication-delay injection described below; they never implement or modify consensus logic.

Raft is provided by `github.com/hashicorp/raft` v1.7.3 [2], wrapped in a thin adapter (`lab/adapters/raft/`) that converts benchmark requests into state-machine commands, invokes `raft.Raft.Apply`, and returns the response. Leader election, log replication, quorum, commitment, and term management are performed by the library. Replicas use BoltDB-backed persistent state in per-replica Docker volumes, with a snapshot threshold of 8,192 entries and 1,024 trailing logs.

The second Raft implementation uses `go.etcd.io/raft` v3.7.0 [1], wrapped in a lab adapter (`lab/adapters/etcdraft/`)

that follows the library’s asynchronous-storage-writes interface: a dedicated append thread batches up to 64 log entries into one write-ahead-log write with a single `fsync`, mirroring HashiCorp Raft’s group commit. The adapter sets `MaxSizePerMsg=4,096` and `MaxInflightMsgs=256` (the library provides no defaults), enables pre-vote to match HashiCorp Raft, and compacts the log below `applied - 1,024` trailing entries, keeping its log in a write-ahead log.

EPaxos uses the upstream `efficient/epaxos` server and master [5], pinned at commit 791b115 and executed with `-e -exec -dreply`. Replica state is in-memory. The upstream implementation fixes the dependency-set size at five; the lab extends this field to support up to nine replicas through a documented wire-format extension that does not alter the protocol phases, quorums, or decision rules. The second EPaxos implementation uses `nvanbenschoten/epaxos` [8], pinned at commit 425bd36, wrapped in a lab adapter (`lab/adapters/nvbepaxos/`) that bridges the library’s gRPC peer transport to the benchmark’s client protocol. EPaxos state is in-memory in both implementations.

3.2.1 Communication-delay injection. The communication-cost family adds a fixed one-way delay to every inter-replica message, implemented in each implementation’s send path: HashiCorp Raft’s transport wrapper sleeps before forwarding `AppendEntries` and `RequestVote`; the `etcd` and `nvb` adapters’ per-peer sender goroutines sleep before writing; and the original EPaxos `genericsmr` send path sleeps before each outbound message (the one place the lab modifies that binary beyond orchestration). The delay is applied per message, so an implementation that sends more messages per request, or serialises its sends, is affected more strongly; the `etcd` adapter’s per-peer sender sleeps on its critical path. The communication-cost results should therefore be read as measurements of each implementation’s send-path structure under added delay, not as a protocol-level latency-sensitivity ranking.

Because the implementations use their native storage and execution mechanisms, the measurements evaluate complete implementations rather than isolated consensus kernels.

3.3 Client Harness and Workload Model

The benchmark client generates requests over a keyspace of 1,000 keys with a configurable read/write composition. The client is closed-loop: one worker per unit of configured concurrency issues its next request only after the previous one completes, so the configured concurrency is the maximum number of in-flight requests. For conflict experiments, a configured conflict ratio determines the probability that a request selects from a shared hot-key set; the realised hot-key fraction is measured from the generated sequence rather than assumed. Workload seeds are recorded for every repetition. Each run begins with a warm-up phase followed by a measured phase; warm-up requests are not included in performance measurements.

3.4 Metrics

For each request, the client records the run identifier, protocol, replica count, workload parameters, concurrency, operation, key, target replica, start and completion timestamps, latency, and success or failure status. The analysis pipeline derives throughput, success rate, and p50/p95/p99 latency. Per-replica resource telemetry is collected at approximately 200 ms resolution: CPU from `cgroup v2 cpu.stat`, network RX/TX from `/proc/<pid>/net/dev`, and resident memory from `/proc/<pid>/status`. Raw samples are retained per replica.

3.5 Testbed Configuration and Parameters

Each performance run consists of a 3-second warm-up followed by a 20-second measured phase, repeated independently ten times. Each replica container is allocated 2.0 CPUs and 512 MB; the client 2.0 CPUs and 512 MB; the coordination service 1.0 CPU and 256 MB. `GOMAXPROCS` is 4 for all replicas. The request timeout is 2,000 ms. Raft uses a 1,000 ms heartbeat timeout, a 2,000 ms election timeout, a snapshot threshold of 8,192 entries, and 1,024 trailing logs. Raft retains its persistent state during a run while EPaxos uses in-memory state; this difference is part of the implementation configuration being measured. Docker volumes are isolated between repetitions so no consensus state carries over.

To examine sensitivity to communication delay, each outbound inter-replica message can be delayed by $d = c + U(0, cj/100)$, where c is a fixed one-way cost in milliseconds (0, 1, 3, 5, 10) and j is a jitter percentage (0, 5, 10, 50, 100, evaluated at the 5 ms cost level). This models application-level message delay rather than a physical network; it does not reproduce queueing, packet loss, or link-level scheduling.

Each repetition uses a fresh Docker Compose project with new containers, networks, and volumes. Before execution the harness verifies the environment is free of leftover containers, networks, volumes, and ports; after execution it verifies teardown. A run that fails isolation checks is recorded as an invalid attempt rather than silently reused. Configurations are scheduled using blocked randomisation: each repetition block contains every configuration exactly once in a seeded shuffled order, and each repetition uses an independent workload seed. Both seeds are recorded.

The principal experiment dimensions are replica count (3, 5, 7, 9), write ratio (0, 10, 50, 90, 100 %), conflict ratio (0, 10, 25, 50, 75, 90, 100 %), client concurrency (1–256), and communication cost (0, 1, 3, 5, 10 ms).

3.6 Experiment Suite

The experiment suite contains seven families, each evaluated with ten independent repetitions:

- **Workload:** write-ratio (0–100 %) and concurrency (1–128) sweeps at three replicas.
- **Scaling:** replica-count sweep (3–9) at 100 % writes and concurrency 32.
- **Conflict:** conflict-ratio sweep (0–100 %) at three replicas, 100 % writes, concurrency 32.

- **Concurrency:** concurrency sweep (1–256) at three replicas, 100 % writes. The Workload and Concurrency families overlap in concurrency (1–128) by design: the Workload family measures the interaction between write ratio and concurrency, while the Concurrency family isolates the concurrency dimension at 100 % writes and extends it to 256, where the throughput difference is largest. The overlap also provides a check on run-to-run variability.
- **Communication cost:** one-way delay sweep (0–10 ms) at three replicas, 100 % writes, concurrency 32, plus a jitter sweep (0–100 %) at the 5 ms level.
- **Election:** Raft leader-election disruption. The leader is terminated and transport to the survivors is isolated for a nominal target of 0–5 failed election attempts; the analysis uses the *measured* failed-election count from the adapters’ pre-vote/vote counters, not the target.
- **Failure:** Raft leader kill, Raft follower kill, and EPaxos replica kill at three replicas, 100 % writes, concurrency 32.

Failure injection is deterministic: the runner kills the selected container with `docker kill` at a configured offset into the measured phase. For Raft leader failure, the current leader is obtained from the raft master and mapped to its container; the client reconnects to the new leader. For EPaxos replica failure, replica 0 is killed and the client moves to the remaining replicas. Recovery intervals are reconstructed from request timestamps and the approximately 200 ms monitoring resolution and are therefore approximate.

3.7 Correctness Validation

The performance experiments are not intended as a formal verification of consensus correctness. A separate correctness harness (`runner correctness`) checks state convergence under the tested execution and failure scenarios. It issues 2,000 deterministic requests, each writing a unique key derived from its request identifier with a deterministic value. Each replica exposes a read-only `GetState` RPC returning a state snapshot and the applied-command count; the harness compares replica states rather than internal logs, so it does not assume that Raft and EPaxos expose equivalent log representations. Each implementation is tested at concurrency 1, 8, and 32 and under the failure scenarios of the failure family. Because Raft persists its log, a restarted Raft replica must recover its committed state; EPaxos uses in-memory state, so a restarted EPaxos replica is evaluated for liveness and subsequent participation. This validation checks state convergence for the tested scenarios; it does not constitute a general linearizability proof.

3.8 Experimental Integrity and Statistical Analysis

Host-level telemetry (load average, available memory) is recorded at approximately 200 ms resolution for every run. Host suspension, scheduling gaps, CPU starvation, and clock

anomalies are detected and recorded. A run flagged as contaminated is excluded from aggregates and replaced so each configuration retains its planned number of observations; the replacement is recorded as a separate attempt and the original remains in the raw dataset.

All aggregates are computed from raw per-run observations. Because a small number of repetitions stalled (Section 4), throughput and latency are summarised by the median of the per-repetition values, with 95 % confidence intervals from non-parametric bootstrap (10,000 resamples). Protocol differences are tested with the exact Mann-Whitney U test and workload-trend hypotheses with the Kruskal-Wallis test; no multiplicity correction is applied. Latency is reported as p50/p95/p99. The election experiment is treated as a Raft-specific recovery characterisation reported descriptively, with no causal interpretation assigned to the measured association.

4 Evaluation

Unless otherwise stated, reported point estimates are medians over ten independent repetitions. Unless otherwise specified, results use three replicas, 100 % writes, and client concurrency 32. All figures are generated from the processed benchmark data by `lab/report/figures.py`. Figures show medians across repetitions, with individual repetition values overlaid where available.

Unless otherwise stated, results describe the evaluated implementations under the experimental configuration of Section 3. Differences should not be interpreted as protocol-level causal effects unless the experiment directly isolates the corresponding mechanism.

4.1 Throughput and Scalability

Figure 1 shows measured throughput as the number of replicas increases. Throughput decreased with replica count for both evaluated implementations. At three replicas, the measured median was 5,292 req/s for EPaxos and 4,351 req/s for Raft. At five replicas, the medians were 5,443 and 3,150 req/s, respectively, while at nine replicas they were 2,094 and 2,113 req/s (Table 1). Relative to the three-replica measurement within this experiment family, the nine-replica median was 60 % lower for EPaxos and 51 % lower for Raft. The ratio of the two medians was 1.22 at three replicas, 1.73 at five, 1.18 at seven, and 0.99 at nine replicas. Thus, the measured throughput difference varied with replica count, and no statistically significant difference was detected at nine replicas.

We report medians rather than means because a small number of repetitions stalled. Across the entire dataset (1,302 runs), eight repetitions completed fewer than 10 % of the requests of their configuration’s median: two Raft runs (0.3 % of Raft runs) and six EPaxos runs (1.0 % of EPaxos runs). Seven of the eight were slow from start to finish with no failed requests, while one EPaxos run at nine replicas showed persistent request timeouts. The two Raft stalls coincided with elevated host load (load average 6.9–8.5 against 3–4

for healthy runs of the same configuration); the six EPaxos stalls occurred at both low and high host load. None of the eight runs was flagged by the host-telemetry contamination rule, so they remain in the dataset. The observed stall counts (2 of 690 Raft runs and 6 of 612 EPaxos runs) were not significantly different (Fisher’s exact test, $p = 0.16$).

A second anomaly affects EPaxos at seven and nine replicas: request timeouts. Every EPaxos run at those sizes except one had 30–250 requests time out at the 2,000 ms client timeout (success rates 98.6–99.9 %), while Raft runs at the same sizes had no timeouts. Each timeout occupies a client worker for the full 2 s, reducing the concurrency available to successful requests. For a closed-loop client with concurrency N , the accounting

$$N \approx X_{\text{ok}} W_{\text{ok}} + X_{\text{timeout}} W_{\text{timeout}},$$

where $X_{\text{timeout}} = N_{\text{timeout}}/20\text{ s}$ and $W_{\text{timeout}} = 2\text{ s}$, is consistent with every EPaxos run at these sizes. The successful requests themselves completed in 5.4–5.6 ms at p50, similar to the three-replica measurements, so the reduced throughput is accounted for by requests occupying workers until the timeout rather than by slower successful requests. The available instrumentation does not establish the specific execution-path mechanism responsible for the timeouts.

The same nominal configuration (three replicas, 100 % writes, and concurrency 32) also occurs independently in several other steady-state experiment families. These repeated measurements provide an indication of run-to-run variability. The four family medians were 4,351, 4,368, 4,483, and 4,441 req/s for Raft, and 5,292, 5,288, 5,177, and 5,250 req/s for EPaxos. The EPaxos medians differed by approximately 2 % across these measurements, and the Raft medians by approximately 3 %. Cross-family variability was therefore of similar magnitude for the two implementations.

For the scaling family, the difference between the implementations was statistically significant at three and five replicas (exact Mann–Whitney U test, $p < 0.001$), while no statistically significant difference was detected at seven ($p = 0.12$) or nine ($p = 0.32$) replicas. The 95 % bootstrap confidence intervals of the medians overlap at seven and nine replicas (Table 1). The measured EPaxos/Raft throughput difference was therefore statistically significant only at the smaller cluster sizes in this experiment.

Figure 2 shows throughput across the client concurrency sweep. Throughput increased with concurrency for both implementations over the tested range. At concurrency 1, the measured medians were 304 req/s for Raft and 175 req/s for EPaxos. At concurrency 256, the medians were 19,333 req/s and 41,077 req/s, respectively. The largest individual observations at concurrency 256 were 21,656 req/s for Raft and 43,681 req/s for EPaxos.

Between concurrency 64 and 256, the Raft median increased from 7,020 to 19,333 req/s, while the EPaxos median increased from 10,712 to 41,077 req/s. Both implementations

Table 1: Throughput (req/s, median of 10 repetitions) by replica count, at 100 % writes and concurrency 32. Brackets show the 95 % bootstrap confidence interval of the median. Medians are used because several repetitions stalled (see Section 4); the mean is lower for both protocols at every replica count.

Replicas	Raft	95 % CI	EPaxos	95 % CI	EPaxos/ Raft
3	4,351	[2,612–4,468]	5,292	[5,118–5,581]	1.22×
5	3,150	[2,771–3,346]	5,443	[4,469–5,611]	1.73×
7	2,590	[1,522–2,608]	3,053	[1,405–4,436]	1.18×
9	2,113	[1,450–2,160]	2,094	[1,386–3,074]	0.99×

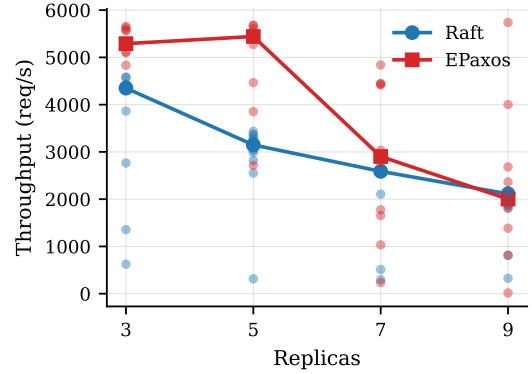


Figure 1: Throughput by replica count at 100 % writes and concurrency 32. Points show individual repetitions; the line marks the median.

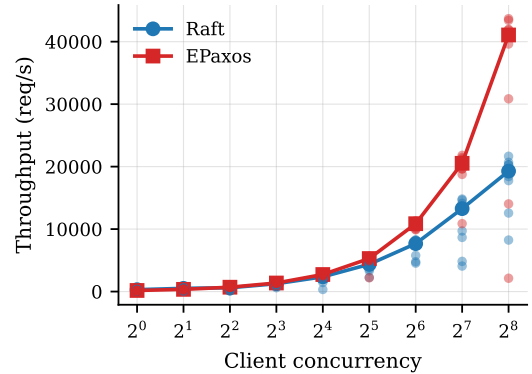


Figure 2: Median throughput by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions.

therefore continued to gain throughput as concurrency increased, although the magnitude of the increase differed between the two implementations.

Figure 3 shows throughput across the write-ratio sweep. EPaxos medians ranged from 5,173 to 5,462 req/s, while Raft

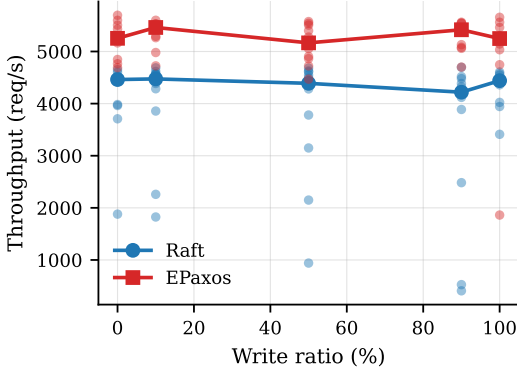


Figure 3: Throughput by write ratio at 3 replicas and concurrency 32. Points show individual repetitions; the line marks the median.

medians ranged from 4,220 to 4,463 req/s. Neither implementation exhibited a monotonic or otherwise consistent throughput trend across the tested write ratios. The highest EPaxos median occurred at 10 % writes, approximately 5 % above its 100 % write measurement, while the highest Raft median occurred at 0 % writes.

The variation across write ratios was not statistically significant. A Kruskal–Wallis test of the five write-ratio groups gave $H = 1.35$ for Raft and $H = 1.69$ for EPaxos, both below the χ^2_4 critical value of 9.49. The measurements therefore do not provide evidence of a systematic throughput effect associated with write ratio under the tested conditions.

Figure 4 shows throughput across the configured conflict-ratio sweep. Median throughput ranged from 4,152 to 4,504 req/s for Raft and from 5,079 to 5,392 req/s for EPaxos. A Kruskal–Wallis test of the seven conflict-ratio groups gave $H = 3.71$ for Raft and $H = 3.18$ for EPaxos, both below the χ^2_6 critical value of 12.59. No monotonic throughput relationship with the configured conflict ratio was observed for either implementation.

4.2 Latency

Figure 5 shows median (p50) latency as the number of replicas increases. At three replicas, the median latency was 6.9 ms for Raft and 5.6 ms for EPaxos. Raft latency increased to 9.2 ms at five replicas and 12.0 ms at seven replicas, reaching 14.6 ms at nine replicas. EPaxos remained within a relatively narrow range of 5.4–5.6 ms across the tested replica counts (Table 2).

The evaluated Raft implementation therefore showed a larger change in median latency with replica count than the evaluated EPaxos implementation under this workload. The Raft p50 increased monotonically with replica count, whereas the EPaxos p50 remained within the 5.4–5.6 ms range.

The p50 values should be read together with the tail. The p99 was 21.4 ms for Raft and 23.8 ms for EPaxos at three replicas, and increased to 26–29 ms for Raft at seven to nine

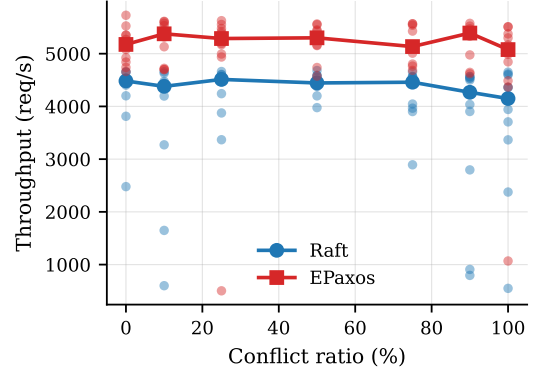


Figure 4: Throughput by conflict ratio at 3 replicas, 100 % writes, and concurrency 32. Points show individual repetitions; the line marks the median.

Table 2: Median (p50) latency in milliseconds by replica count, at 100 % writes and concurrency 32. Values are the median of the ten per-repetition p50 values.

Replicas	Raft p50 (ms)	EPaxos p50 (ms)
3	6.9	5.6
5	9.2	5.5
7	12.0	5.4
9	14.6	5.5

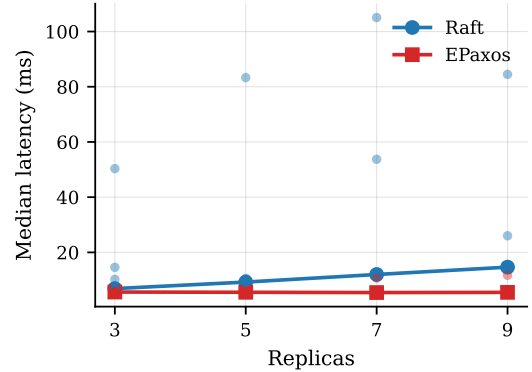


Figure 5: Median (p50) latency by replica count at 100 % writes and concurrency 32. Points show individual repetitions; the line marks the median.

replicas while remaining at 14–16 ms for EPaxos. The median of the per-repetition p50 values reduces the influence of the stalled repetitions described earlier.

Figure 6 shows median latency across client concurrency. At concurrency 1, median latency was 2.8 ms for Raft and 5.5 ms for EPaxos. At concurrency 32, the corresponding

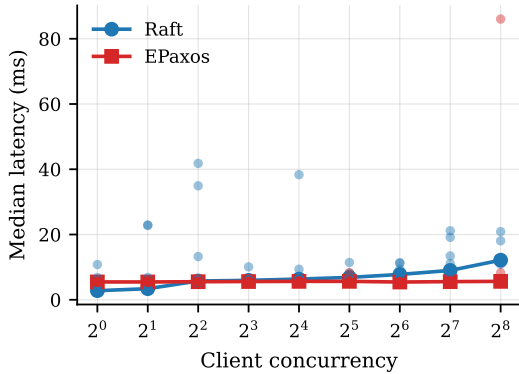


Figure 6: Median (p50) latency by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions; the line marks the median.

values were 6.9 ms and 5.6 ms. At concurrency 256, they were 12.1 ms and 5.6 ms.

The relative latency ordering therefore changed with concurrency. Raft had the lower median at concurrency 1, while EPaxos had the lower median at concurrency 32 and above. At concurrency 256, the measured p50 was 5.6 ms for EPaxos and 12.1 ms for Raft.

As with throughput, the same nominal configuration (three replicas, concurrency 32) is measured by several experiment families. With medians, the p50 was consistent across families: Raft measured 6.9 ms in the scaling family and 6.9 ms in the concurrency family, while EPaxos measured 5.6 ms in both.

For a closed-loop client, Little’s law relates mean latency to concurrency and throughput as $W = N/X$. The reported p50 is the median rather than the mean, so it need not equal this quantity. At concurrency 256, the Raft p50 of 12.1 ms was close to N/X ($256/19,333 \approx 13.2$ ms), while the EPaxos p50 of 5.6 ms was below N/X ($256/41,077 \approx 6.2$ ms). This difference is compatible with a response-time distribution whose median is below its mean. We report p50, p95, and p99 in the processed dataset and use p50 here as the central-tendency summary.

At seven and nine replicas, the EPaxos p50 of 5.4–5.5 ms was substantially below N/X ($32/2,094$ – $3,053 \approx 10.5$ – 15.3 ms). As described in Section 4, this difference is accounted for by the request timeouts at those sizes: each timeout occupies a worker for the full 2 s, and the closed-loop concurrency accounting including the timeout term is consistent with every EPaxos run. Raft runs at the same sizes had no timeouts, and their measured N/X values were closer to their reported p50 latencies.

4.3 Resource Utilisation

Figure 7 summarises the per-replica resource measurements. Across the evaluated runs, mean per-replica CPU utilisation was 35.8 % for Raft and 35.0 % for EPaxos. Mean peak per-replica RSS was 26.4 MB for Raft and 46.8 MB for EPaxos.

Table 3: Failure behaviour at 3 replicas, 100 % writes, concurrency 32. Gap is the largest interval with no successful write completion after failure injection.

Failure	Protocol	Gap (s)	Failed req.
Leader kill	Raft	1.81–4.74	2,169–4,133
Follower kill	Raft	0.02–0.04	0
Replica kill	EPaxos	0.02–0.09	10–106

Mean per-replica network rates were 441 kB/s RX and 443 kB/s TX for Raft, compared with 408 kB/s RX and 392 kB/s TX for EPaxos.

Figure 8 examines the distribution of CPU utilisation in the nine-replica scaling experiment. Raft per-replica means ranged from 16.3 % to 29.4 %. The replica serving as leader during a given repetition generally consumed less CPU than the followers. Labelling each run by the leader reported at the start of its measured phase, the leader/follower CPU ratio was 0.61–0.64 at three replicas, 0.70 at five replicas, and 0.73–0.75 at seven and nine replicas, using the medians over the ten repetitions.

This pattern is consistent with the request path of the evaluated HashiCorp Raft implementation, in which the leader sends AppendEntries and waits for quorum responses while followers receive, persist, and apply replicated log entries. The measurements do not isolate the contributions of persistence, state-machine application, or communication processing to the observed CPU difference. They therefore establish that the leader did not form the CPU hotspot in the evaluated implementation, but do not by themselves identify the individual processing components responsible for the difference.

4.4 Failure Recovery Behaviour

Failure outcomes differed by failure type (Table 3, Figure 9). Killing the Raft leader produced a write-availability gap of 1.81–4.74 s and 2,169–4,133 failed requests. Killing a Raft follower produced a gap of 0.02–0.04 s with no failed requests. Killing an EPaxos replica produced a gap of 0.02–0.09 s with 10–106 failed requests.

All runs completed successfully according to the experiment harness. The reported failed requests correspond to requests issued during the transition and are therefore distinct from run-level failure.

The longer availability gap following Raft leader termination coincided with the configured Raft election and request-retry behaviour. In contrast, the tested follower-failure case did not require replacement of the Raft leader, while the EPaxos replica-failure case continued with the remaining replicas. These results describe the tested failure scenarios and should not be interpreted as representative of all leader-based or leaderless implementations.

The election experiment provides a more controlled characterisation of Raft-specific recovery under repeated election disruption. The measured number of failed elections ranged from 0 to 13 across the 60 runs. The availability gap

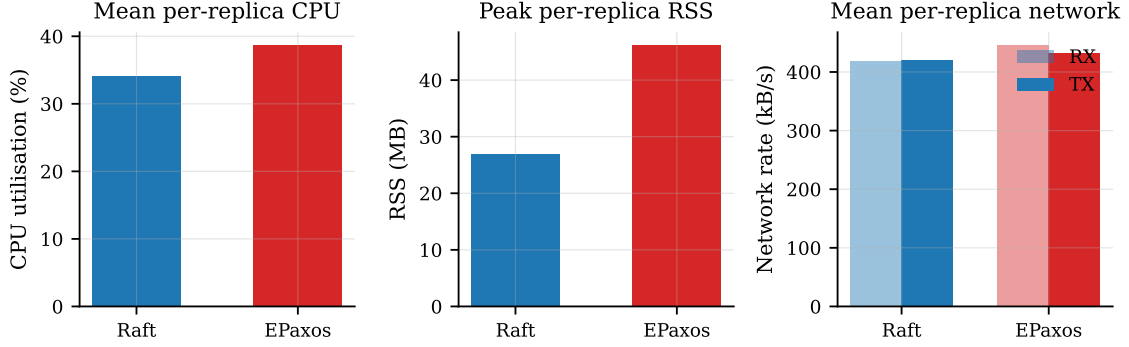


Figure 7: Mean per-replica CPU utilisation, peak per-replica RSS, and mean per-replica network rates, aggregated over the evaluated runs.

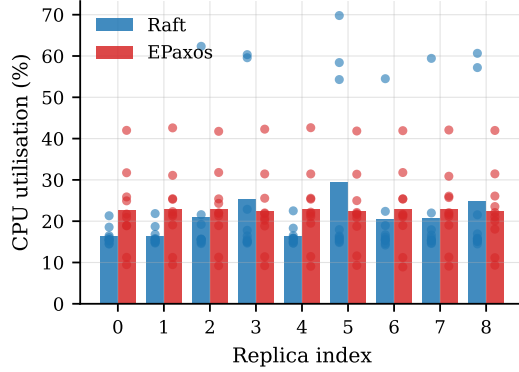


Figure 8: Per-replica CPU utilisation in the 9-replica scaling runs. Bars show means over repetitions; points show individual repetitions.

was strongly associated with the measured election count ($r = 0.90$, $p < 10^{-4}$), but this relationship is partly mechanical because the isolation interval itself increases with the configured disruption duration.

To examine recovery independently of the imposed isolation interval, the analysis also measures the time from the end of the isolation window to the first successful request. This post-isolation recovery interval was more variable and showed a weaker positive association with the measured failed election count ($r = 0.41$, $p = 0.001$, $n = 60$). This result is reported as an observed association rather than as evidence of a causal relationship.

4.5 Correctness Validation

The correctness harness described in Section 3 checks state convergence under the tested workloads and failure scenarios. All 18 tests passed (Table 4): every replied request was applied exactly once on every replica, the surviving replicas agreed on the final state, the restarted Raft replicas recovered the committed state from their persistent logs, and requests

Table 4: Correctness-validation results: 2,000 deterministic requests per test at 3 replicas, 100 % writes. Every replied request was applied exactly once on every replica, surviving replicas agreed on the final state, and requests issued after each fault were processed normally.

Implementation	Sequential	c=8	c=32	Fault
Raft (HashiCorp)	PASS	PASS	PASS	follower/leader
Raft (etcd)	PASS	PASS	PASS	follower/leader
EPaxos (original)	PASS	PASS	PASS	replica
EPaxos (nvb)	PASS	PASS	PASS	replica

issued after each fault were processed normally. Because the correctness workload assigns a unique key to each request, this is primarily a state-convergence check rather than an independent verification of operation-order semantics.

4.6 Communication Cost

The communication-cost family evaluates sensitivity to additional application-level one-way message delay. It should therefore be interpreted as a controlled communication-condition experiment rather than as a direct simulation of a physical network.

Figure 11 shows mean throughput at each delay level with zero jitter for four implementations. The implementations exhibited substantially different sensitivity to the injected delay. HashiCorp Raft decreased from 4,502 req/s at zero delay to 1,280 req/s at 10 ms, a 72 % reduction. nvb EPaxos decreased from 4,346 to 833 req/s, an 81 % reduction, while the original EPaxos implementation decreased from 5,527 to 221 req/s, a 96 % reduction. The etcd Raft implementation showed a different response: its throughput decreased from 6,420 req/s at zero delay to 375 req/s at 1 ms and 45 req/s at 10 ms.

The measured ordering therefore differed across implementations under added communication delay. The 95 %

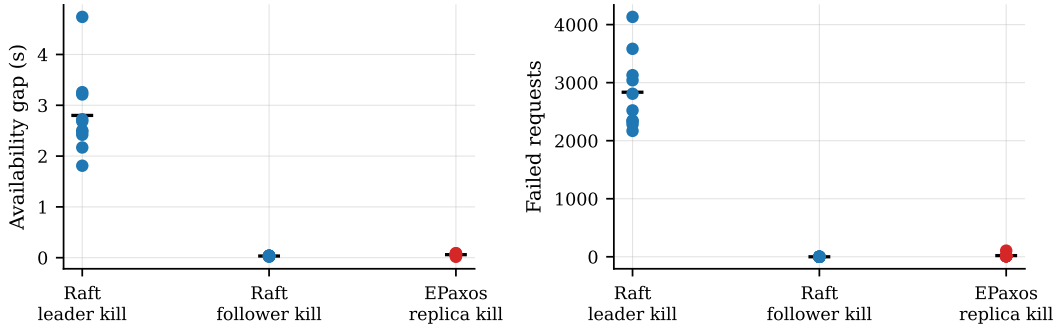


Figure 9: Write-availability gap and failed requests by failure type at 3 replicas, 100 % writes, and concurrency 32. Points show individual repetitions; the bar marks the mean.

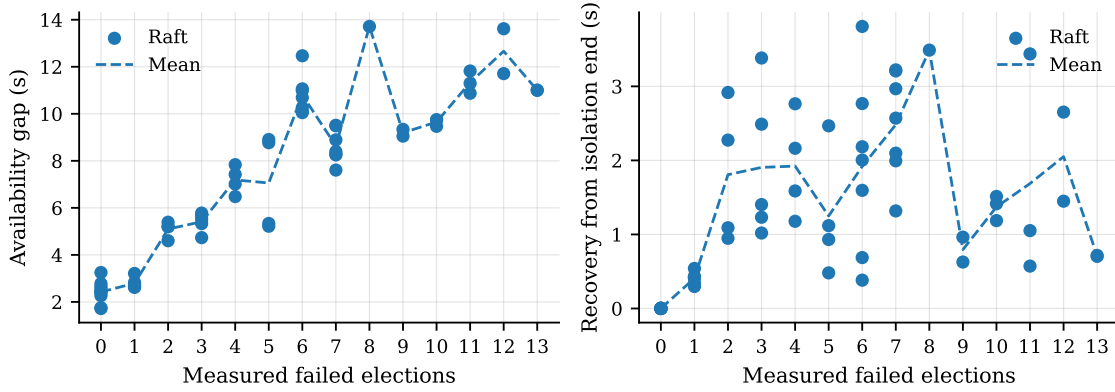


Figure 10: Election-failure recovery in Raft at 3 replicas, 100 % writes, and concurrency 32. Left: write-availability gap versus the measured number of failed elections. Right: recovery time from the end of the isolation window to the first successful request. Points show individual repetitions; the dashed line marks the mean per value.

confidence intervals of the means were separated between the evaluated implementations at every non-zero delay level.

Median latency increased with the injected delay for all four implementations. At 10 ms, the measured p50 values were 22.6 ms for HashiCorp Raft, 36.2 ms for nvb EPaxos, 125.9 ms for original EPaxos, and 703.4 ms for etcd Raft (Figure 12). The implementation ordering observed for latency at this condition corresponded to the throughput ordering.

The jitter sweep produced a smaller change than the corresponding increase in fixed delay. At a 5 ms base delay, increasing jitter from 0 to 100 % reduced throughput from 2,316 to 1,760 req/s for HashiCorp Raft (24%), from 1,432 to 1,082 req/s for nvb EPaxos (24%), from 479 to 305 req/s for original EPaxos (36%), and from 87 to 59 req/s for etcd Raft (32%).

Under the implemented jitter model, 100 % jitter increases the maximum additional delay from zero to 5 ms and increases the mean injected delay from 5 to 7.5 ms. The measured throughput changes are therefore consistent with an increase

in average injected delay. The experiment does not separate the effect of delay variance from the corresponding change in mean delay.

4.7 Summary of Findings

The measurements support the following observations under the evaluated conditions:

- Throughput decreased with replica count for both implementations (nine-replica median 60 % below three-replica for EPaxos, 51 % for Raft). The EPaxos/Raft median ratio was 1.22 at three replicas, 1.73 at five, 1.18 at seven, and 0.99 at nine, where no statistically significant difference was detected.
- Eight of 1,302 repetitions stalled (two Raft, six EPaxos), motivating the use of medians. EPaxos at seven and nine replicas also showed request timeouts (30–250 per run) that account for its reduced throughput via the closed-loop concurrency accounting of Section 4.

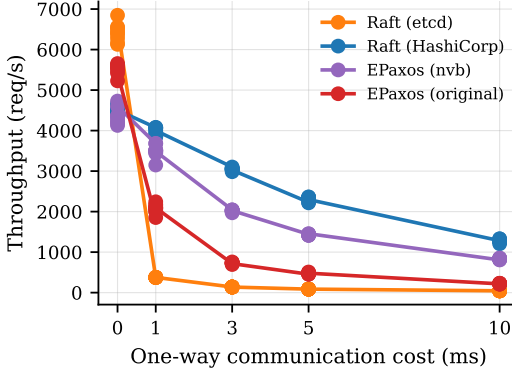


Figure 11: Mean throughput by application-level one-way communication delay at 3 replicas, 100 % writes, and concurrency 32 for the four evaluated implementations.

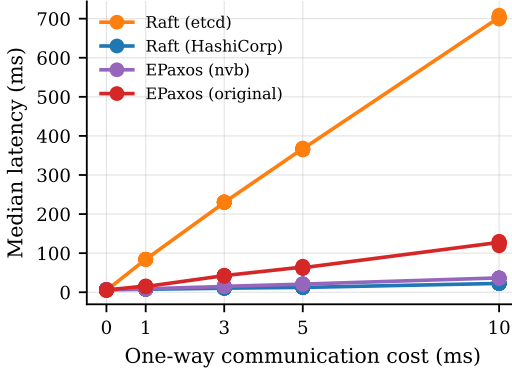


Figure 12: Median (p50) latency by application-level one-way communication delay at 3 replicas, 100 % writes, and concurrency 32 for the four evaluated implementations.

- At concurrency 256, the medians were 41,077 req/s for EPaxos and 19,333 req/s for Raft, with p50 latency 5.6 ms against 12.1 ms.
- Write ratio and conflict ratio produced no systematic throughput trend (Kruskal–Wallis $H \leq 3.71$ against critical values of 9.49 and 12.59).
- Raft leader termination produced an availability gap of 1.81–4.74 s, whereas Raft follower and EPaxos replica termination produced gaps below 0.1 s.
- Across the scaling configurations, the Raft leader/follower CPU ratio remained below 1.0 (0.61–0.64 at three replicas, 0.70 at five, 0.73–0.75 at seven and nine), so the leader did not form the CPU hotspot in the evaluated HashiCorp Raft implementation.
- Under 10 ms of injected one-way delay, throughput fell by 72–96 % across the four implementations, and

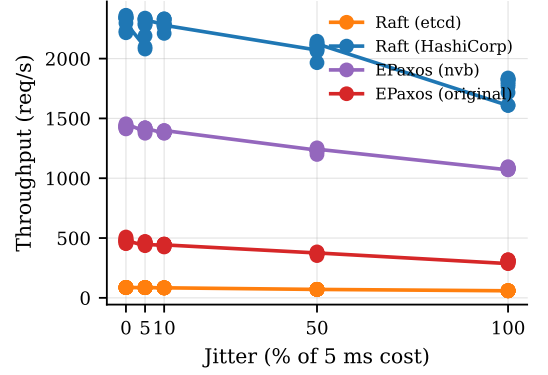


Figure 13: Mean throughput by jitter at a 5 ms application-level one-way delay, 3 replicas, 100 % writes, and concurrency 32 for the four evaluated implementations.

etcd Raft collapsed by 94 % at 1 ms, showing that implementation choice materially changes the measured performance relationship.

5 Discussion

5.1 RQ1: Throughput and Latency Across Workloads

RQ1 asks how the measured performance of the evaluated leader-based and leaderless implementations changes with replica count, client concurrency, conflict rate, and read/write composition. Replica count and concurrency produced the clearest differences. With three replicas, EPaxos achieved 5,292 req/s compared with 4,351 req/s for Raft, and with five replicas the corresponding medians were 5,443 and 3,150 req/s. The differences were statistically significant at both configurations ($p < 0.001$), whereas no significant difference was detected at seven or nine replicas. At the latter replica counts, EPaxos throughput was reduced by request timeouts, while the p50 latency of successful requests remained between 5.4 and 5.6 ms. Raft p50 latency increased from 6.9 ms at three replicas to 14.6 ms at nine replicas.

Client concurrency produced a similar divergence. At concurrency 256, EPaxos reached 41,077 req/s compared with 19,333 req/s for Raft, with p50 latencies of 5.6 and 12.1 ms, respectively. This behaviour is consistent with the evaluated EPaxos configuration accepting proposals at multiple replicas, whereas requests in the evaluated Raft configuration are directed through the elected leader. The experiment does not isolate this mechanism from other implementation-level costs.

In contrast, varying conflict rate and read/write composition did not produce a systematic throughput trend in the evaluated range. Kruskal–Wallis tests were not significant for either implementation in the conflict-ratio experiment or the write-ratio experiment. Thus, under the workload and consensus path used here, the measured throughput differences

were more sensitive to replica count and client concurrency than to the tested changes in conflict or read/write composition. These results describe the evaluated implementations and should not be interpreted as establishing that conflict rate or read/write composition is irrelevant to the underlying protocols.

5.2 RQ2: Failure and Communication Behaviour

RQ2 examines behaviour under replica and leader failures and under injected communication delay. Failure behaviour depended strongly on which replica failed. Killing the Raft leader produced availability gaps of 1.81–4.74 s and 2,169–4,133 failed requests, whereas killing a Raft follower produced gaps of only 0.02–0.04 s with no failed requests. EPaxos replica failures produced gaps of 0.02–0.09 s with 10–106 failed requests. The difference between Raft leader and follower failures is therefore visible in the measured availability behaviour of the evaluated implementation. The failed-election count was strongly associated with the observed availability gap ($r = 0.90$), although this relationship is partly mechanical because the isolation duration increased with the failure target. The residual recovery association ($r = 0.41$) is therefore reported descriptively rather than as evidence of a causal relationship.

Injected communication delay produced substantial performance changes, but the magnitude depended strongly on the concrete implementation. At 10 ms of one-way injected delay, throughput decreased by 72–96 % across the four implementations. The etcd Raft implementation decreased from 6,420 req/s at zero delay to 375 req/s at 1 ms and 45 req/s at 10 ms. This behaviour is associated with the per-message sender serialisation exposed by the application-level delay injection, so the result characterises the interaction between the implementation’s send path and the experimental condition rather than establishing a protocol-level latency-sensitivity ranking.

Jitter produced a smaller effect than the fixed communication cost. With a 5 ms base delay, throughput changes at 100 % jitter were in the 24–36 % range across the evaluated implementations. The observed effect is consistent with jitter changing the distribution of injected message delay while retaining a comparable mean-delay scale. Together, the failure and communication experiments show that availability and performance under adverse conditions depend on both the failure role and the communication mechanisms exercised by the concrete implementation.

5.3 RQ3: Implementation-Level Sources of Performance Difference

RQ3 asks how much of the measured performance difference between Raft and EPaxos is attributable to the concrete implementation rather than to the consensus algorithm family. The four-implementation experiment shows that the relationship between the two protocol families changes substantially

with implementation and operating condition. At zero injected delay, the measured throughputs were 6,420 req/s for etcd Raft, 5,527 req/s for the original EPaxos, 4,502 req/s for HashiCorp Raft, and 4,346 req/s for nvb EPaxos. The baseline ordering therefore did not follow the leader-based versus leaderless classification.

The differences became larger under communication delay. At 10 ms, HashiCorp Raft measured 1,280 req/s, nvb EPaxos 833 req/s, original EPaxos 221 req/s, and etcd Raft 45 req/s. Implementations belonging to the same protocol family consequently exhibited substantially different responses to the same communication condition. An implementation with higher baseline throughput also did not necessarily retain the highest throughput after delay was introduced.

These observations indicate that the measured performance relationship contains contributions from the complete implementation stack, including request processing, message handling, persistence, scheduling, and transport behaviour. The evaluated Raft implementations differ in storage and message-handling mechanisms, while the EPaxos implementations also differ in their transport and request-processing paths. Consequently, the observed differences cannot be attributed to the consensus algorithm family alone when comparing these concrete implementations. The results instead support an implementation-sensitive interpretation: measured performance depends jointly on consensus structure, implementation, workload, replica count, concurrency, communication conditions, and failure mode.

6 Threats to Validity

6.1 External Validity

All replicas run on a single host over a bridge network, so the measurements contain no wide-area latency, no real packet loss, and no multi-machine failure domains. The host is a WSL2 environment with 12 physical cores, placing a virtualisation layer between the containers and the hardware. Cluster sizes are 3–9, the keyspace is 1,000 keys, commands are uniform in size, and the measured phase is 20 s. The results therefore describe short, uniform, single-host runs rather than long-lived deployments with skewed access patterns, large payloads, or geographically distributed replicas.

The configured CPU quotas exceed the physical core count at the larger cluster sizes (nine replicas at 2.0 CPUs each plus client and master request 21 CPUs against 12 cores), but the measured host load average stayed at 3.8–5.1 across the scaling family and the per-replica CPU means imply roughly 4–5 cores of actual demand, so the host was not CPU-saturated and the throughput decline with replica count is not explained by host saturation. Each replica is allocated 2.0 CPUs while `GOMAXPROCS=4`, so the Go scheduler parallelism exceeds the container quota; the resulting interaction with cgroup throttling is part of the measured environment.

Reads are routed through consensus in both protocols and no Raft-specific read optimisation is enabled, so the read comparison is of equivalent consensus-facing paths rather than each system’s fastest possible read path. Both systems

are measured with identical CPU and memory allocations, which provides a controlled comparison but not a resource-tuned deployment.

Persistence is a source of asymmetry between the protocols: both Raft implementations persist their log (BoltDB, or a write-ahead log with batched fsync) while both EPaxos implementations keep state in memory, and the Raft adapters compact their logs (snapshot threshold 8,192 entries, 1,024 trailing logs), so snapshot and compaction work is part of the steady-state Raft path. The latency and throughput differences therefore include the cost of persistence and log maintenance, which is a property of the evaluated implementations, not of the protocols themselves. The EPaxos implementation is additionally modified to support the dependency-set sizes required by the 7- and 9-replica configurations; results at those sizes should not be interpreted as isolating the cost of the EPaxos protocol from the cost of this extension.

The communication experiment injects delay in the application-level outbound message path rather than through a physical network emulator, so it does not reproduce queueing, packet loss, or link-level scheduling, and the jitter experiment changes the mean delay when jitter is increased. The failure and election experiments use three replicas, so their recovery times characterise that configuration only; the failure scenarios are also not symmetric (the Raft leader-failure experiment removes the normal request endpoint, whereas the follower- and replica-failure experiments remove a non-central replica). The Raft leader-recovery interval depends on the configured heartbeat and election-timeout parameters and on client timeout and reconnection behaviour.

The correctness harness verifies state convergence under the tested workloads and restart scenarios but does not constitute a formal verification of consensus correctness or establish general linearizability; the unique-key workload does not exercise conflicting operations on the same state in a way that would expose every possible ordering violation.

6.2 Statistical Conclusion Validity

Each configuration is measured ten times, and aggregates are reported as medians with non-parametric bootstrap 95% confidence intervals. Ten repetitions leave substantial uncertainty: measuring the same nominal configuration in four steady-state families produced medians that differed by 2–3% for both protocols, and the bootstrap intervals overlap in several comparisons whose medians differ, such as at seven and nine replicas. We therefore treat overlapping comparisons as inconclusive rather than as evidence of a small but established effect.

The suite contains many configurations and pairwise comparisons without a global multiplicity correction, so an isolated statistically detectable difference should not be interpreted as strong evidence for a general effect. The election correlation analysis requires particular caution: the measured failed-election count is an outcome of the recovery process and is mechanically related to the injected isolation duration, and the 60 runs are grouped by six isolation targets (ten runs

each), so the observations are not fully independent and the reported p -values are optimistic. The correlation should be read as a descriptive association, not as a hypothesis test with 60 independent observations.

The 20s measurement window limits the characterisation of long-term steady-state behaviour, and the closed-loop client means the configured concurrency is a maximum number of in-flight requests rather than an independently controlled arrival rate. Outliers within clean runs are not removed; runs identified as affected by environmental interference are investigated separately and are not excluded solely because their performance is inconvenient for the comparison.

7 Conclusion

This paper presented an experimental comparison of Raft and EPaxos as representative leader-based and leaderless consensus approaches. Rather than treating a single throughput measurement as representative of a consensus protocol, the evaluation examined throughput, latency, replica scaling, client concurrency, failure recovery, resource utilisation, and communication sensitivity under controlled variations in workload and system conditions.

The workload and scaling experiments show that the measured performance relationship depends on the operating condition. Increasing the replica count reduced throughput for both evaluated implementations, while increasing client concurrency produced a substantially larger throughput increase for EPaxos at high concurrency. The evaluated Raft configuration routes normal requests through an elected leader, whereas EPaxos permits proposal initiation at multiple replicas. This structural difference is consistent with the larger throughput response observed for EPaxos under high concurrency, but the measurements do not establish that distributed proposal processing is the sole cause of the difference. The read/write and conflict experiments further show that workload composition does not necessarily produce a simple or monotonic throughput response.

The failure experiments identify a distinct operational consequence of the leader-centred request path. Failure of the Raft leader produced an availability gap of 1.81–4.74s, whereas failure of a Raft follower produced gaps below 0.1s in the evaluated three-replica configuration. EPaxos replica failure likewise produced short interruptions in the tested scenario. These results show that failure behaviour depends on which replica is removed and how the request-processing path is structured. The election experiment further characterised the Raft recovery process, with the measured number of failed election attempts strongly associated with the observed availability gap. These measurements describe the tested recovery configuration rather than protocol-independent recovery bounds.

The implementation-sensitivity experiments provide the strongest qualification to direct protocol comparisons. The

evaluated Raft and EPaxos implementations exhibited substantially different absolute performance and different responses to injected communication cost, including substantial differences between implementations of the same protocol family. Persistence, request processing, transport handling, scheduling, and adapter behaviour can therefore materially affect measured throughput and latency without changing the underlying consensus algorithm. Absolute benchmark results should consequently be interpreted as properties of the evaluated implementation and environment rather than as intrinsic performance constants of Raft or EPaxos.

Taken together, the results do not establish a universal performance ordering between leader-based and leaderless consensus. Instead, they show that the observed relationship depends jointly on consensus structure, implementation, workload, replica count, client concurrency, communication conditions, and failure mode. The evaluated Raft configuration exposes the concentration and failure dependency associated with a leader-centred request path, while the evaluated EPaxos configuration exposes a different trade-off through distributed proposal placement and dependency coordination.

The study is limited by its single-host containerised environment, application-level rather than physical network-delay injection, tested cluster sizes, closed-loop workload model, and the specific implementations evaluated. The correctness experiments additionally establish state convergence under the tested workloads and failures rather than formal consensus correctness or general linearizability. These limitations restrict direct generalisation to production deployments and larger or geographically distributed systems.

Nevertheless, the experimental methodology provides a reproducible basis for studying consensus implementations under controlled changes in workload and system conditions. Future work can extend the comparison to additional independent implementations, larger clusters, multi-host deployments, network-level emulation, detailed profiling, and workloads that exercise stronger ordering and contention conditions. Such extensions would help further distinguish protocol-level effects from implementation-specific behaviour.

Acknowledgments

Generative AI tools were used during the preparation of this work for assistance with benchmark and laboratory code development, language editing, proofreading, and drafting and revision of manuscript text. The author independently defined the research questions, experimental design, overall methodology, and interpretation of the study. Experimental execution, data analysis, and verification of the reported results were conducted and checked by the author. AI-generated suggestions were reviewed and revised by the author, who is responsible for the final content and technical claims of the manuscript.

References

- [1] etcd Authors. 2025. etcd: Distributed Reliable Key-Value Store (raft package). <https://github.com/etcd-io/etcd/tree/main/raft>. Version v3.7.0.
- [2] HashiCorp. 2025. HashiCorp Raft: A Raft Consensus Protocol Implementation in Go. <https://github.com/hashicorp/raft>. Version v1.7.3.
- [3] Heidi Howard and Richard Mortier. 2020. Paxos vs Raft: Have We Reached Consensus on Distributed Consensus?. In *7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC 20)*. ACM, 1–9. doi:10.1145/3380787.3393681
- [4] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (1998), 133–169. doi:10.1145/279227.279229
- [5] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There Is More Consensus in Egalitarian Parliaments. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 13)*. ACM, Farmington, PA, 358–372. doi:10.1145/2517349.2522738
- [6] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [7] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. EPaxos Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 613–632. <https://www.usenix.org/conference/nsdi21/presentation/tollman>
- [8] Nathan van Benschoten. 2025. EPaxos: Egalitarian Paxos in Go. <https://github.com/nvanbenschoten/epaxos>. Commit 425bd36.
- [9] Robbert van Renesse, Nicolas Schiper, and Fred B. Schneider. 2015. Vive La Différence: Paxos vs. Leaderless Replication. In *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing (PODC 15)*. ACM, 242–251. doi:10.1145/2767386.2767419